Docker Startup Script

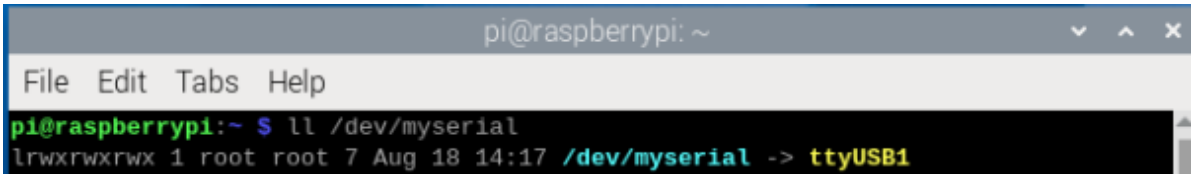
1. Check the Bound Devices

The A1 Pro robot car has a total of three bound devices. After a successful boot, run the following commands to check whether the binding was successful.

- Check the binding status of the ROS expansion board.

```
ll /dev/myserial
```

If the output matches the figure below, the binding was successful.



- Check the binding status of the LiDAR.

```
ll /dev/rplidar
```

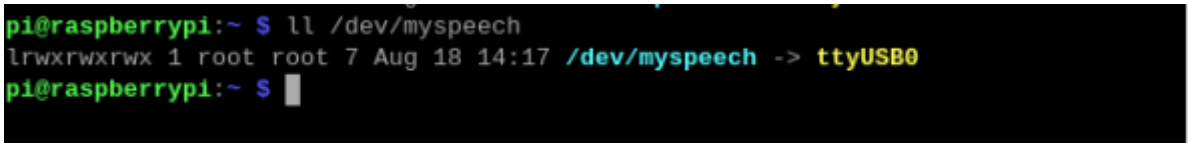
If the output matches the figure below, the binding was successful.



- Check the binding status of the AI voice broadcast module.

```
ll /dev/myspeech
```

If the output matches the figure below, the binding was successful.



If you purchased the depth camera, you can also run the following command to check whether the depth camera was successfully bound.

```
ll /dev/as_camera
```

If the output matches the figure below, the binding was successful.

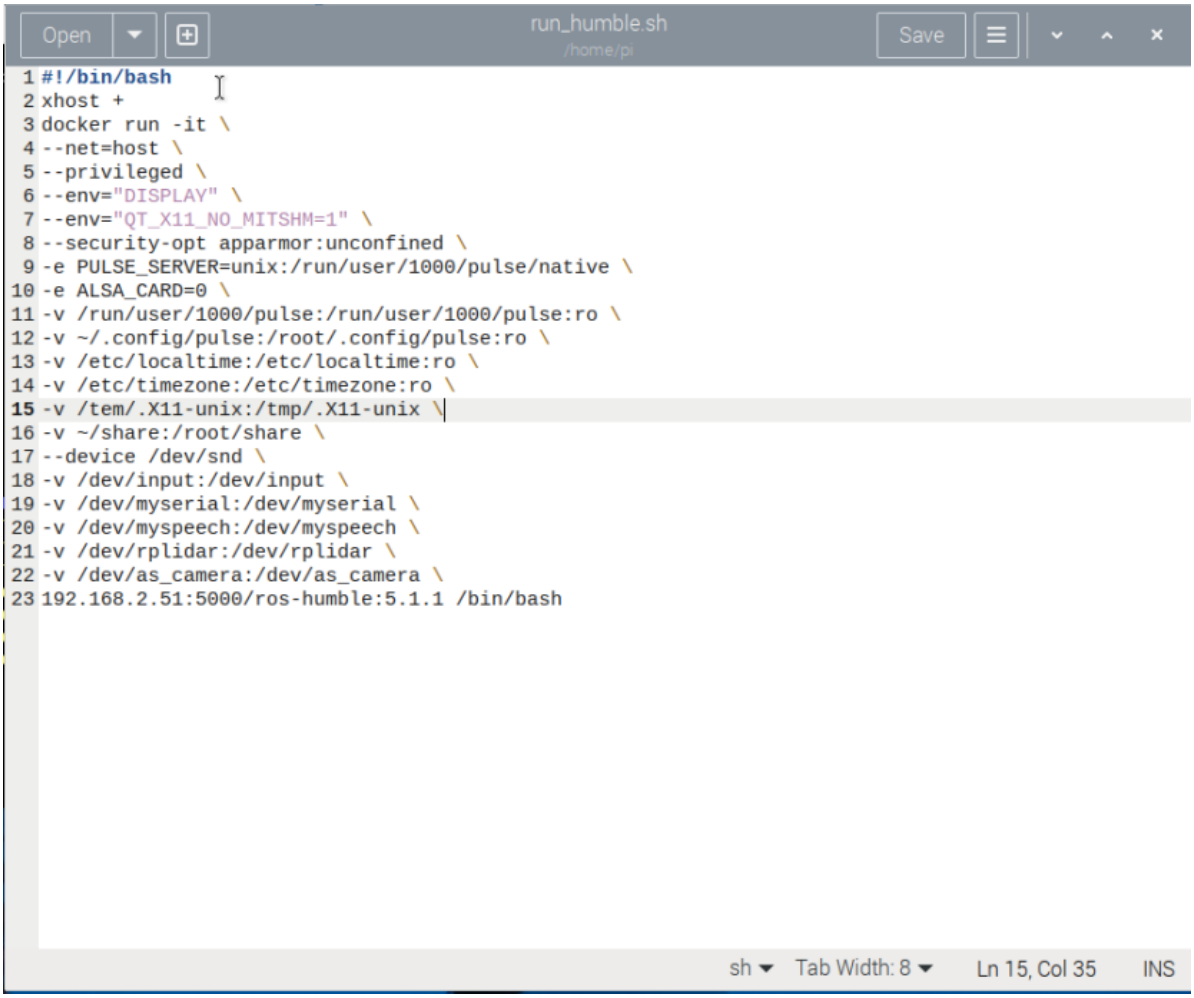


2. View the Script Definition

On the Raspberry Pi or Jetson Nano host, run the following command to open the Docker script.

```
sudo gedit run_humble.sh
```

After pressing Enter, you can see the script contents.

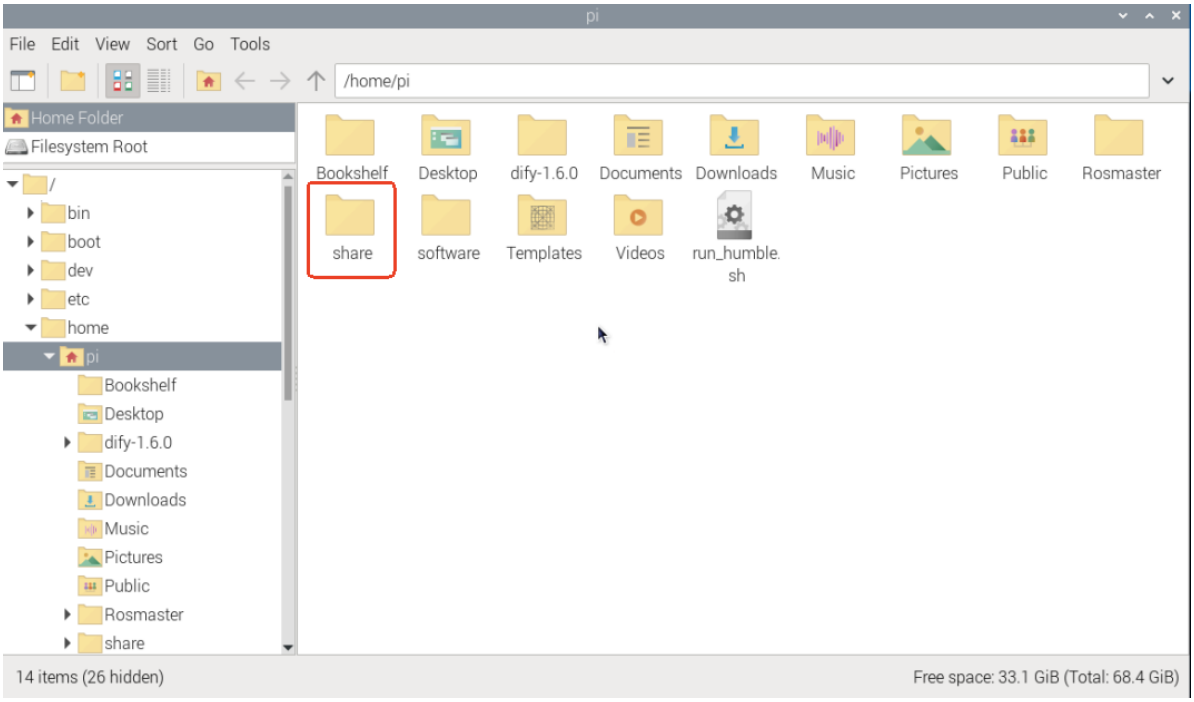


Below is a brief explanation of each line of the Docker script. Its main function is to start a Docker container and map the hardware devices connected to the Raspberry Pi into the container, so that the peripherals connected to the Raspberry Pi's interfaces can be used freely in Docker.

```
#!/bin/bash
xhost +             # allow the container to access the host's X11 display
service, i.e. provide visualization capabilities to Docker
docker run -it \
--net=host \       # the container uses the host's network stack directly
--privileged \     # grant access to all hardware devices
--env="DISPLAY" \   # pass the visualization display environment variable
--env="QT_X11_NO_MITSHM=1" \
--security-opt apparmor:unconfined \   # disable the security policy when
visualization is not possible
-e PULSE_SERVER=unix:/run/user/1000/pulse/native \  # PulseAudio service path
-e ALSA_CARD=0 \
-v /run/user/1000/pulse:/run/user/1000/pulse:ro \  # mount the audio socket read-
only
-v ~/.config/pulse:/root/.config/pulse:ro \
-v /etc/localtime:/etc/localtime:ro \   # synchronize the local time
```

```
-v /etc/timezone:/etc/timezone:ro \
-v /tem/.X11-unix:/tmp/.X11-unix \
-v ~/share:/root/share \   # shared folder between the host and Docker
--device /dev/snd \   # access the host's sound card
-v /dev/input:/dev/input \ # mount all input devices (mouse/keyboard)
-v /dev/myserial:/dev/myserial \ # mount the ROS expansion board
-v /dev/myspeech:/dev/myspeech \ # mount the AI module serial port
-v /dev/rplidar:/dev/rplidar \  # mount the LiDAR
-v /dev/as_camera:/dev/as_camera \  # mount the depth camera
192.168.2.51:5000/ros-humble-a1:1.0.6 /bin/bash   # select the Docker image to
run
```

Note that you should check whether the mounted peripherals are recognized correctly, and whether the Docker container to be run exists. A shared folder will be created on the host; you can place the files or photos you want to transfer into Docker into this folder.

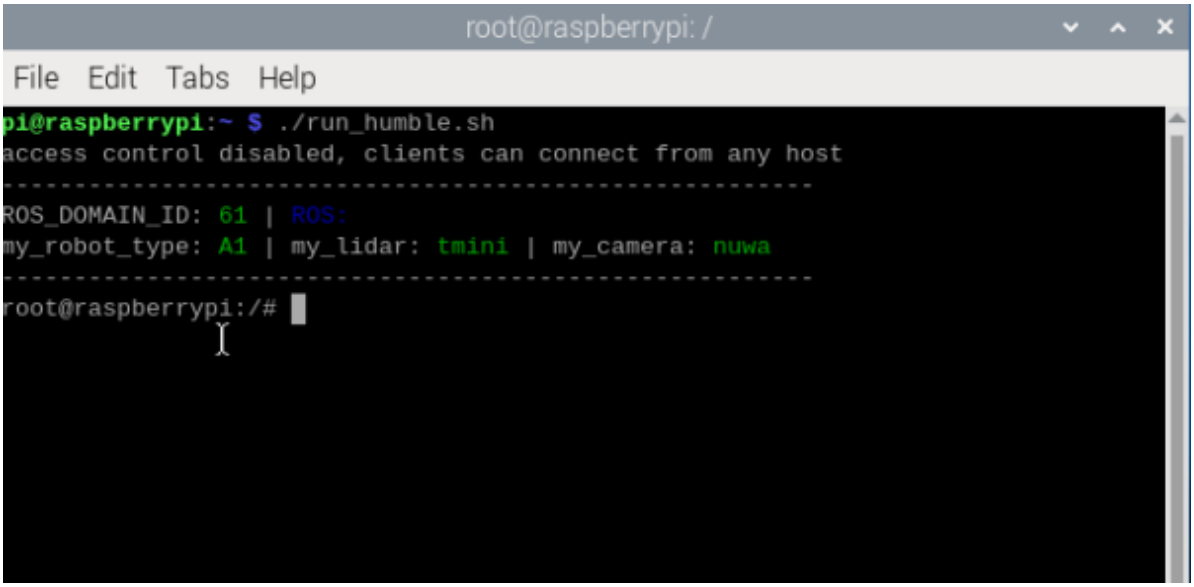


3. Run the Docker Script

When we remotely connect via VNC using the IP address shown on the OLED small screen, we can open a terminal and run the following command to enter Docker.

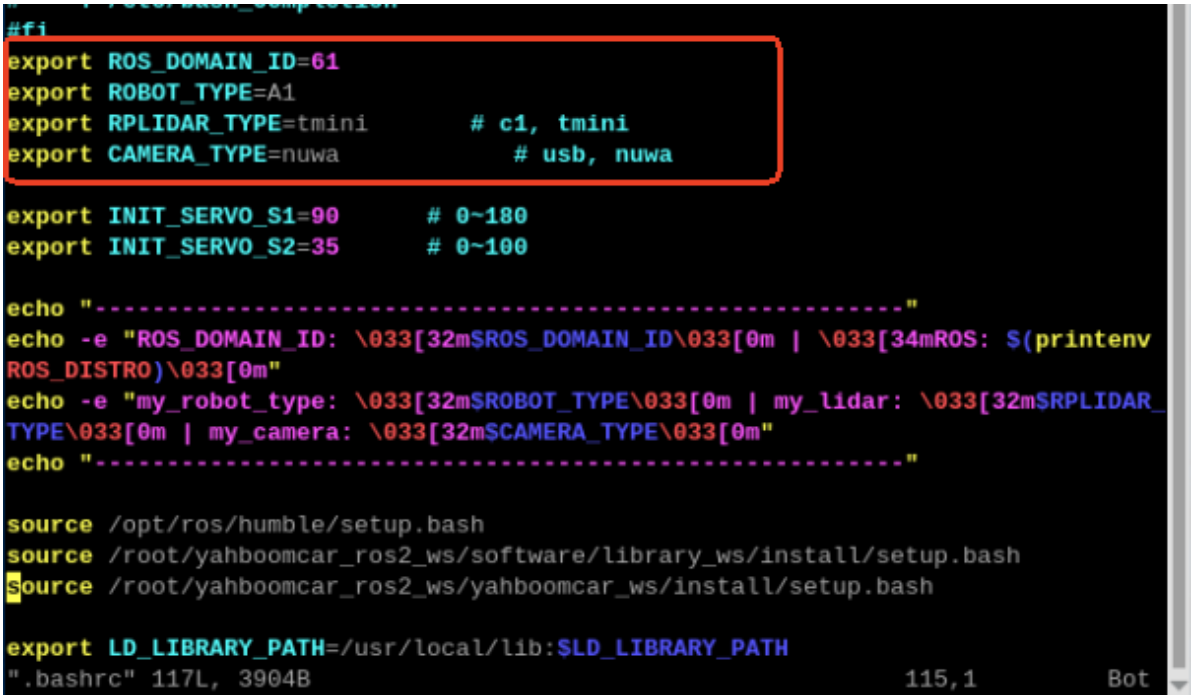
```
./run_humble.sh
```

If successful, the output is as follows.



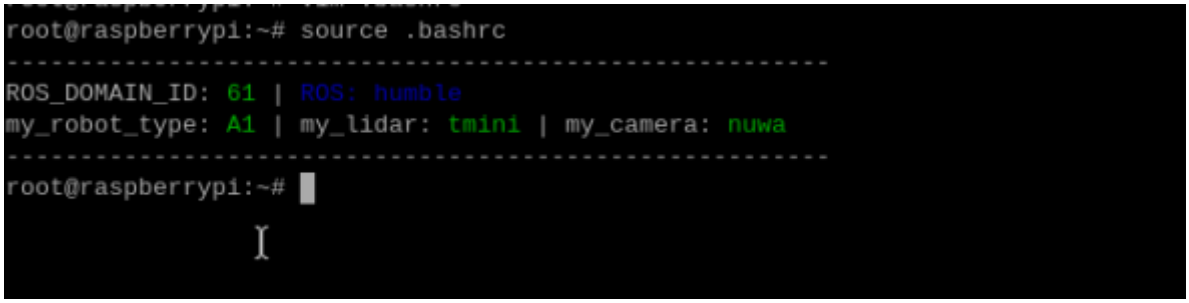
Because the robot is available in two camera models and two different LiDAR models, the factory system has pre-configured the routines for multiple devices. However, since the product cannot be identified automatically, the machine type and LiDAR model need to be set manually. Enter the following in the terminal.

```
root@raspberrypi:/# cd
root@raspberrypi:~# vim .bashrc
```



After configuring your own peripherals, save and exit vim, then run:

```
source .bashrc
```



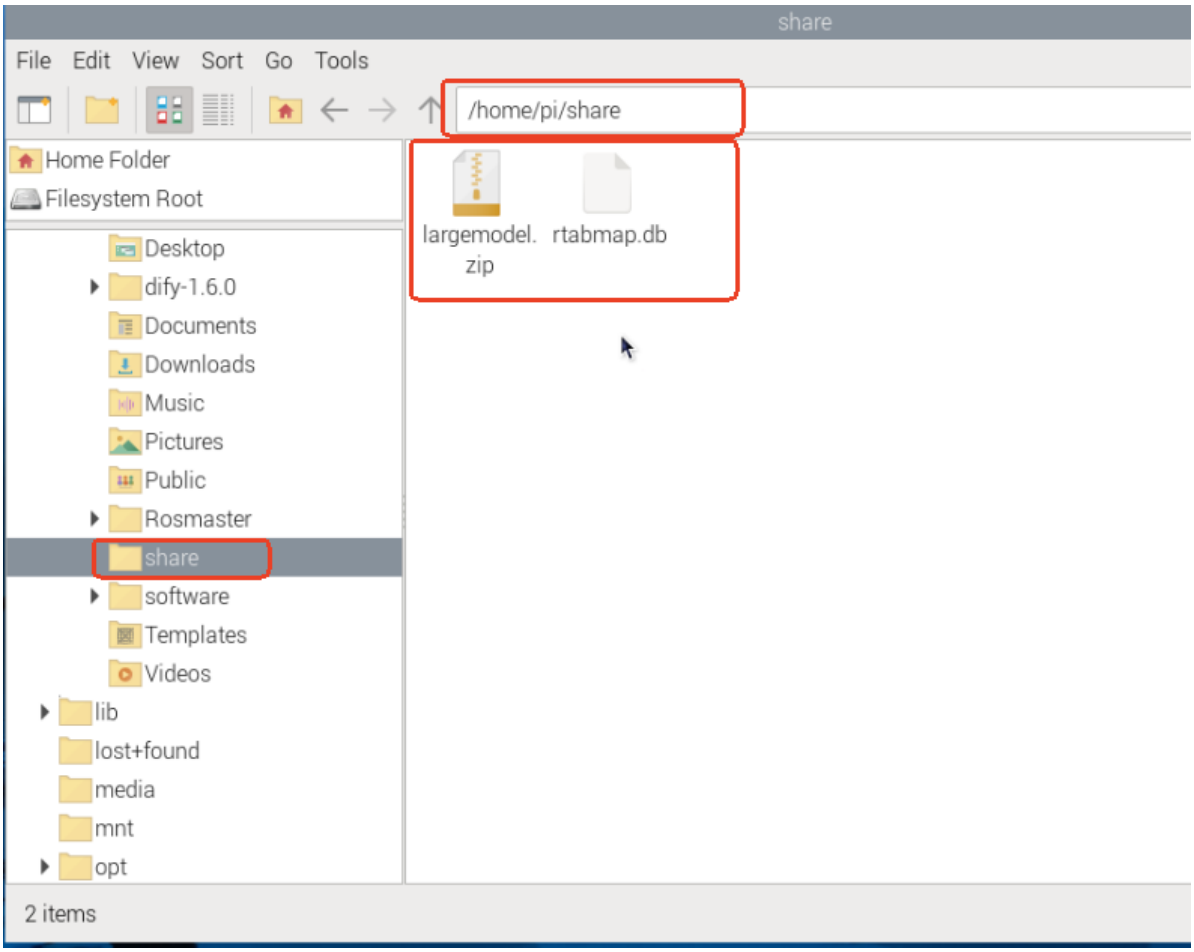
You can now see the currently configured robot model, LiDAR type, and camera type of the modified robot car.

For the shared folder, you can run the following commands to view it.

```
root@raspberrypi:~# cd share/
root@raspberrypi:~/share# ls
```



We can see that there is a .db file and a compressed archive. On the Raspberry Pi side, the same files also exist.



Based on this principle, files can be transferred between the host and Docker.